

Universidad ORT Uruguay

Facultad de Ingeniería

Evidencia de Clean Code y de la aplicación de TDD

Obligatorio 1 – Diseño de Aplicaciones 2: FarmeORT

Juan Ferreiro – 328344

Sebastián Luciardi – 274754

Andrés Bustamante – 302420

Grupo M6C – Tutores: Alexander Wieler y Franco Bonavita

2026

Repositorio: https://github.com/IngSoft-DA2/M6CD-ID-274754-328344-302420

Declaración de autoría

Nosotros, Juan Ferreiro, Sebastián Luciardi y Andrés Bustamante, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el Obligatorio 1 de Diseño de Aplicaciones 2;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.

Juan Ferreiro, Sebastián Luciardi y Andrés Bustamante – 08/10/2026

Abstract

Este documento explica cómo y por qué el backend de FarmeORT se construyó con TDD y siguiendo Clean Code. El equipo aplicó una estrategia Outside-In: cada funcionalidad empezó por un test del controller y bajó al servicio, al dominio y al repositorio, con un commit por etapa del ciclo RED, GREEN y REFACTOR (516, 517 y 114 commits). En el cuerpo se muestra un ciclo completo con las capturas de sus commits; la evidencia de las demás funcionalidades está en los anexos. Los 1.046 tests pasan, todos los assemblies superan el 90 % de líneas y de ramas que exige el CI y, excluyendo lo que no es testeable, la cobertura es del 100 %. El foco está en el porqué: por qué Outside-In, por qué esos dobles de prueba, qué ganó el proyecto con TDD y por qué el código cumple Clean Code.

Índice

1. Estrategia de TDD: Outside-In 5

1.1 Cómo se aplicó 5

1.2 Por qué Outside-In 5

1.3 Dobles de prueba y aislamiento 6

1.4 Proyectos de prueba 7

2. Nuestra forma de trabajo: RED, GREEN y REFACTOR 8

2.1 Un commit por etapa 8

2.2 Un ciclo completo, paso a paso 8

2.3 Evidencia en las funcionalidades prioritarias 12

2.4 Corrección de errores guiada por pruebas 13

2.5 Pull Requests, revisiones y GitHub Actions 13

2.6 Uso de IA 14

3. Beneficios de TDD en FarmeORT 15

4. Ejecución de las pruebas y cobertura 16

4.1 Ejecución 16

4.2 Cómo se midió la cobertura 16

4.3 Resultado 17

4.4 Calidad de las pruebas más allá de la cobertura 18

5. Clean Code: por qué lo cumplimos 19

5.1 Reglas automáticas 19

5.2 Cómo se aplica cada tema y qué se gana 19

5.3 Un ejemplo: el cálculo de ELO 21

6. Referencias bibliográficas 22

Anexo A. Evidencia de TDD por funcionalidad prioritaria 23

A.1 Inscripción a un torneo 23

A.2 Demás funcionalidades prioritarias 24

Anexo B. Cobertura sin exclusiones 26

Anexo C. Clean Code en detalle 28

C.1 Nombres y funciones 28

C.2 Comentarios y formato 28

C.3 Objetos, manejo de errores y límites 28

C.4 Pruebas unitarias y clases 29

C.5 Diseño emergente y auditoría de legibilidad 29

1. Estrategia de TDD: Outside-In

Los datos de este documento salen del repositorio del equipo [1], rama develop, en el commit 3ac010e (merge del PR #44, el último integrado a develop el 07/10/2026). Cada hash citado es un link al commit en GitHub, y las decisiones se identifican con su código en la bitácora del repositorio (por ejemplo, D-68).

1.1 Cómo se aplicó

Cada funcionalidad se empezó por el borde de la API y se bajó capa por capa: Controller, IServicio, Servicio, Dominio, IRepositorio y Repositorio. El test del controller, con un mock del servicio, fija el código de estado, el header Location y que el servicio reciba al usuario del token; así aparece la firma que necesita IServicio. Los tests del servicio, con mocks de los repositorios, cubren una precondición o un efecto cada uno y definen qué métodos necesita cada IRepositorio. Las reglas que dependen solo de una entidad se prueban sobre ella sin dobles, y los repositorios se prueban con EF Core sobre SQLite en memoria cuando la consulta tiene lógica propia.

1.2 Por qué Outside-In

El obligatorio se define por endpoints y reglas de negocio, y el contrato de la API se acordó antes de escribir código. Empezar por el controller convierte ese contrato en un test desde el primer commit y hace que cada capa se construya para quien la usa: las interfaces IServicio e IRepositorio las define el test de la capa consumidora, no la que las implementa. Así no aparecen métodos de repositorio «por si acaso» ni entidades con datos que ningún caso de uso pide. Además encaja con la arquitectura (D-34): como la WebAPI solo conoce IServicios y los servicios solo conocen IRepositorios, cada capa se puede probar antes de que exista la de abajo. La Tabla 1 resume la comparación con la alternativa descartada.

Tabla 1. Outside-In frente a Inside-Out

Criterio	**Outside-In (elegida)**	**Inside-Out (descartada)**
Punto de partida	El requisito tal como lo ve el cliente de la API	Las entidades y la base de datos
Qué se construye	Solo lo que el endpoint necesita	Lo que se supone que se va a necesitar

Criterio	Outside-In (elegida)	Inside-Out (descartada)
Quién define las interfaces	El test de la capa que las usa	La capa que las implementa
Cuándo se contrasta con el contrato	Desde el primer commit	Al final, cuando se integra el controller
Riesgo principal	Tests atados a las interacciones por el uso de mocks	Código sin uso (YAGNI) y contratos que no calzan

La contracara de Outside-In es que los tests de controller y de servicio usan dobles, y un test con mocks puede quedar atado a cómo está escrito el código en lugar de a qué hace. Se mitigó de tres formas: los mocks solo verifican los efectos que pide la regla (sección 1.3), las reglas que pertenecen a una entidad bajan al dominio y se prueban con objetos reales, y los repositorios se prueban contra una base real. El orden se ve en el historial: en la inscripción a un torneo (PR #16) el primer RED fue el del controller (ecc5f4b, 19:25), después el del servicio (87a5577, 19:34) y luego el del dominio (dd52c24, 19:42). La secuencia completa está en el Anexo A.

1.3 Dobles de prueba y aislamiento

Cada decisión sobre los dobles de prueba responde a un riesgo concreto. Moq se usa en modo estricto (MockBehavior.Strict) y con Setup sobre valores exactos: si el servicio llama a un repositorio que el test no previó, o con otro argumento, el test falla. Eso obliga a pensar qué necesita cada caso de uso y avisa cuando un cambio agrega una consulta inesperada. It.IsAny solo aparece para verificar que algo no ocurrió (Times.Never). Los repositorios que solo proveen datos son stubs, y los efectos se verifican con Verify sobre el estado exacto del objeto guardado, para que el test pruebe la regla y no la implementación.

La fecha (TimeProvider, D-18) y el azar del empate de ELO (ISorteo, D-51) se inyectan porque un test que depende del reloj o de Random no es repetible. Los repositorios se prueban con la clase BaseDeDatosEnMemoria, que crea el esquema con el mismo FarmeORTContext sobre SQLite en memoria: mockear DbSet solo probaría que se llama a LINQ, mientras que una base real prueba lo que de verdad se rompe (filtros, orden con desempate, índices únicos y guardados atómicos). Como un test de controller no ejecuta los filtros, cada endpoint protegido tiene además un test que lee por reflexión sus atributos de autenticación y autorización (D-15). Por último, ningún test usa otro método del objeto bajo prueba para preparar o verificar, así una falla apunta a un solo método.

1.4 Proyectos de prueba

Hay un proyecto de prueba por cada assembly con lógica, cada uno referencia solo el assembly que prueba y los tipos internos se exponen con InternalsVisibleTo (ver Tabla 2). Se usan MSTest 4.0.2, Moq 4.21.0, EF Core Sqlite 10.0.11 y coverlet.collector 6.0.2.

Tabla 2. Proyectos de prueba y cantidad de tests

Proyecto de prueba	**Qué prueba**	**Tests**
FarmeORT.Dominio.Tests	Reglas de las entidades, cálculo de ELO y emparejamiento. Sin Moq: el único doble es SorteoPreparado.	218
FarmeORT.Servicios.Tests	Casos de uso con mocks estrictos: un test por precondición y por efecto.	329
FarmeORT.Repositorios.Tests	Consultas, filtros, paginado, guardados atómicos y concurrencia con EF Core SQLite en memoria.	185
FarmeORT.WebAPI.Tests	Controllers, filtros de autenticación, autorización y excepciones, y validaciones de los DTOs.	301
FarmeORT.Fabrica.Tests	Que cada servicio de la API se resuelva con lifetimes válidos.	13
Total		**1.046**

Que cada proyecto referencie solo su assembly evita que un test de servicio pase «de casualidad» porque ejecutó código real de repositorio, y hace que la cobertura de cada assembly salga de las pruebas pensadas para él.

2. Nuestra forma de trabajo: RED, GREEN y REFACTOR

2.1 Un commit por etapa

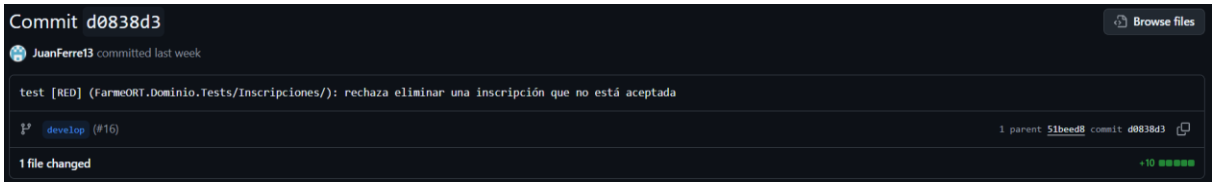
Cada etapa es un commit con el formato "tipo [ETAPA] (ruta): descripción". El commit test [RED] agrega un test que falla por la razón esperada (un assertion o la excepción que falta) y, si hace falta, solo la firma mínima del tipo o método nuevo, que lanza NotImplementedException; como el .editorconfig marca como error los parámetros sin usar, esa firma los descarta explícitamente hasta el GREEN (D-05). El commit feat [GREEN] o fix [GREEN] agrega el código mínimo para pasar, y refactor [REFACTOR] mejora la estructura sin cambiar el comportamiento. En las funcionalidades prioritarias se hizo un test por commit RED, y las variantes de un mismo caso van con DataRow. El historial tiene 516 commits RED, 517 GREEN y 114 REFACTOR, sobre 1.354 commits sin contar los merges.

Separar las etapas en commits tiene tres ventajas. Cada línea de producción queda asociada al test que la pidió, y el historial lo demuestra. Los pasos son chicos: si algo falla, el error está en el último cambio. Y el REFACTOR, al ir solo, prueba que la mejora de estructura no cambió el comportamiento, porque los tests no se tocan y siguen en verde; si fuera junto con el GREEN no se podría distinguir una cosa de la otra. Cuando no había nada que mejorar no se inventó un refactor; por eso hay menos REFACTOR que GREEN.

2.2 Un ciclo completo, paso a paso

El ciclo elegido es de la inscripción a un torneo, una de las funcionalidades prioritarias. Cuando se falla una batalla, la inscripción del perdedor pasa a Eliminada. Inscripcion.Eliminar() ya existía y Cancelar() ya rechazaba una inscripción que no estuviera Aceptada, pero Eliminar() todavía permitía eliminar una inscripción cancelada. Los tres commits se hicieron en poco más de un minuto (de 19:48 a 19:49 del 01/10/2026), y las capturas muestran que son consecutivos: el padre de cada uno es el anterior (51beed8 → d0838d3 → 3adb163 → 24c7ea5), y cada uno cambia un solo archivo.

RED: d0838d3 test [RED] (FarmeORT.Dominio.Tests/Inscripciones/): rechaza eliminar una inscripción que no está aceptada



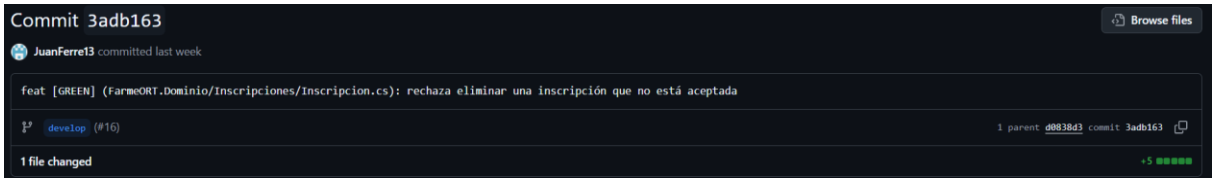

```
[TestMethod]
public void Eliminar_InscripcionCancelada_LanzaInscripcionNoActivaSinCambiarla()
{
    var inscripcion = new Inscripcion(_torneo, _jugador, Ahora);
    inscripcion.Cancelar();

    Assert.ThrowsExactly<InscripcionNoActivaException>(inscripcion.Eliminar);
    Assert.AreEqual(EstadoInscripcion.Cancelada, inscripcion.Estado);
}
```

Ilustración 1. Commit RED d0838d3 en GitHub y el test que agrega

El commit agrega únicamente el test Eliminar_InscripcionCancelada_LanzaInscripcionNoActivaSinCambiarla y no toca código de producción. Falla por la razón esperada: Eliminar() no lanza ninguna excepción, así que falla el Assert.ThrowsExactly, no la compilación ni la configuración. El nombre dice qué método se prueba, en qué escenario y qué se espera, y el segundo assert fija una parte de la regla que es fácil olvidar: si se rechaza, el estado no cambia. La captura muestra que el commit solo suma 10 líneas en un archivo, el de tests.

GREEN: 3adb163 feat [GREEN] (FarmeORT.Dominio/Inscripciones/Inscripcion.cs): rechaza eliminar una inscripción que no está aceptada



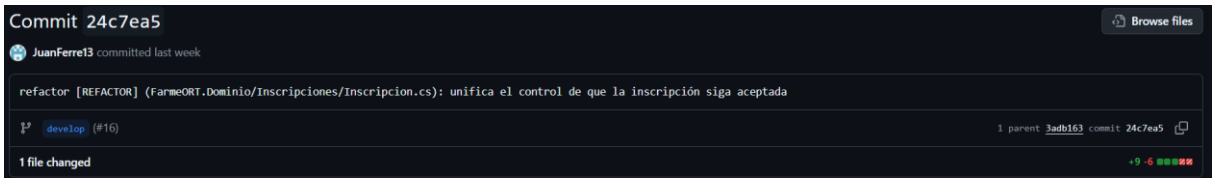

```
// La usa el fallo de una batalla: el perdedor queda fuera del torneo.
public void Eliminar()
{
    if(Estado != EstadoInscripcion.Aceptada)
    {
        throw new InscripcionNoActivaException();
    }

    Estado = EstadoInscripcion.Eliminada;
}
```

Ilustración 2. Commit GREEN 3adb163 en GitHub y el método después del cambio

El GREEN agrega cinco líneas: el mismo if que ya tenía Cancelar(). Se copió a propósito, porque en esta etapa la prioridad es volver a verde con el menor cambio posible y no diseñar (la captura muestra +5 líneas en Inscripcion.cs). La duplicación queda a la vista y se resuelve en la etapa siguiente.

REFACTOR: 24c7ea5 refactor [REFACTOR] (FarmeORT.Dominio/Inscripciones/Inscripcion.cs): unifica el control de que la inscripción siga aceptada




```
// No controla el estado del torneo: también se usa al cancelar un torneo En curso.
public void Cancelar()
{
    ValidarAceptada();

    Estado = EstadoInscripcion.Cancelada;
}

// La usa el fallo de una batalla: el perdedor queda fuera del torneo.
public void Eliminar()
{
    ValidarAceptada();

    Estado = EstadoInscripcion.Eliminada;
}

// Cancelada y Eliminada son estados finales.
private void ValidarAceptada()
{
    if(Estado != EstadoInscripcion.Aceptada)
    {
        throw new InscripcionNoActivaException();
    }
}
```

Ilustración 3. Commit REFACTOR 24c7ea5 en GitHub y los métodos después del cambio

El REFACTOR extrae el control a ValidarAceptada(), que usan Cancelar() y Eliminar(), con un comentario que explica el porqué (Cancelada y Eliminada son estados finales). No cambia ningún test: los de Cancelar y Eliminar siguen en verde, y esa es la garantía de que el comportamiento es el mismo; el commit toca solo Inscripcion.cs (+9 y −6 líneas). La regla

queda en un solo lugar y en la entidad que tiene la información (Information Expert), no en el servicio que la usa.

2.3 Evidencia en las funcionalidades prioritarias

La letra del obligatorio [2] marca con (*) cinco funcionalidades cuya evolución debe verse en los commits. Cada una se desarrolló en su propia rama y entró a develop por un Pull Request con merge commit, sin squash, para conservar los commits de cada etapa. La Tabla 3 cuenta esos commits dentro de la rama de cada funcionalidad. Después de su PR siguieron con ciclos propios: concurrencia (PR #28 y #31), auditoría de legibilidad (PR #30 y #32) rutas de la revisión docente (PR #33) y el refactor de Clean Code (PR #43, sección 5.2).

Tabla 3. Commits por etapa en la rama de cada funcionalidad prioritaria

Funcionalidad (*)	**Rama y PR**	**Autor**	**RED**	**GREEN**	**REFACTOR**
1.5 Inscripción a un torneo	feature/inscripciones, PR #16	Juan Ferreiro	55	55	3
2.2 Catálogo de skills	feature/catalogo-skills, PR #9	Andrés Bustamante	46	46	8
1.8 Lanzar una skill	feature/batallas-lanzamientos, PR #14	Sebastián Luciardi	44	44	2
1.10 Cálculo de ELO	feature/calculador-elo, PR #15	Andrés Bustamante	4	4	1
1.9 Fallar una batalla	feature/fallos, PR #22	Sebastián Luciardi	18	18	1
2.5 Generar rondas	feature/rondas, PR #25	Andrés Bustamante	38	38	6
2.5 Finalizar el torneo	feature/finalizacion-torneo, PR #29	Andrés Bustamante	16	16	1

El Anexo A tiene el historial de la rama de inscripciones y ciclos representativos de cada una de las demás funcionalidades.

2.4 Corrección de errores guiada por pruebas

Los errores se corrigieron con el mismo ciclo. El servicio rechazaba la inscripción a un torneo En curso con "cupo completo" en lugar de "el torneo no está abierto": RED 12a2bb6 y GREEN 02d0120 (D-67). Y Enum.TryParse aceptaba el texto "1" como rol Organizador: RED 4f86e7e y GREEN cb29be6 (D-68).

Escribir primero el test que reproduce el error confirma que se entendió la causa y deja una prueba que impide que el error vuelva.

2.5 Pull Requests, revisiones y GitHub Actions

Todo el código entró a develop por Pull Request (44 PR integrados al 07/10/2026), con la plantilla obligatoria del curso, merge commit (squash y rebase están deshabilitados en el repositorio) y los tres checks obligatorios en verde: Build, Test y Analysis. Los workflows son los de la configuración obligatoria del curso y corren en los self-hosted runners de los integrantes. La Tabla 4 muestra cuántos PR abrió y aprobó cada integrante.

Tabla 4. Pull Requests por integrante

Integrante (cuenta de GitHub)	**PR de los que es autor**	**PR que revisó y aprobó**
Juan Ferreiro (JuanFerre13)	25	11
Andrés Bustamante (AndresBustamanteORT)	10	8
Sebastián Luciardi (Seba0512)	9	24

Las revisiones dejan comentarios técnicos y, cuando corresponde, piden cambios antes de aprobar: en los PR #7 (registro de jugadores) y #17 (ranking) el revisor pidió cambios y aprobó después de verificar las correcciones.

Que los checks sean obligatorios para mergear hace que las reglas no dependan de la memoria de nadie: un PR con un test en rojo, con un assembly por debajo del 90 % o con un cambio de formato pendiente no entra a develop. Y como los commits se conservan (merge sin squash), quien revisa puede leer los RED en orden y entender qué reglas se agregaron antes de mirar el código.

2.6 Uso de IA

El equipo usó Claude Code (modelo Claude Opus 5.5) como herramienta para acortar los tiempos de trabajo y cumplir con los plazos de entrega: ayudó a revisar el código contra la letra y la configuración del curso, y a reproducir las mediciones de cobertura. El equipo definió el alcance, revisó todo lo generado y puede explicar cada decisión.

3. Beneficios de TDD en FarmeORT

Esta sección resume los beneficios que se observaron en este proyecto, cada uno con su evidencia.

Refactorizar con una red de seguridad. Antes de la entrega el equipo revisó todo el código para que un tercero lo entienda con el menor esfuerzo (D-68, PR #30 y #32) y reemplazó construcciones poco conocidas por if explícitos en todos los assemblies. Sin tests, un cambio así es riesgoso; con la suite completa en verde después de cada paso se hizo sin cambiar el comportamiento, y los dos errores que aparecieron se corrigieron con su propio ciclo RED y GREEN. Lo mismo vale para los 114 commits REFACTOR, entre ellos los 19 del refactor de Clean Code del PR #43, que no modificaron ningún test (sección 5.2).

Los errores no vuelven. Cada error corregido dejó un test que lo reproduce (sección 2.4). Si alguien reintroduce el problema, el check Test lo frena antes del merge.

El diseño sale de quien usa el código. Las interfaces de los repositorios tienen solo los métodos que algún servicio necesitó: ContarAceptadas apareció como firma en el commit RED 2904e6b porque el test del cupo la pedía. No hay repositorios genéricos ni métodos sin uso.

Dependencias inyectables por necesidad. Para probar que empieza el jugador de mayor ELO y que un empate se decide al azar, el azar tuvo que inyectarse (ISorteo); para probar fechas, el reloj (TimeProvider). Ese mismo diseño permite cambiar esas piezas sin tocar el código que las usa, igual que con ICalculadorElo e IEstrategiaEmparejamiento.

Los tests documentan la letra. Los nombres de ServicioInscripcionesTests, como Inscribir_CupoDeAceptadasCompleto_LanzaTorneoCompletoSinGuardar, se leen como la lista de precondiciones de la letra para inscribirse. Para saber qué hace el sistema ante un caso alcanza con buscar su test.

Revisiones más simples. Quien revisa un PR lee los RED en orden y sabe qué reglas se agregaron; cada GREEN muestra cuánto código hizo falta para cada una.

Costo asumido. Escribir primero el test hace más lento el primer avance de cada funcionalidad, y los mocks estrictos obligan a actualizar tests cuando cambia una interfaz (en el GREEN 2f46811 hubo que configurar ContarAceptadas en los tests que ya existían). El equipo lo aceptó porque ese tiempo se recupera en cada refactor y en cada error que no llegó a develop.

4. Ejecución de las pruebas y cobertura

4.1 Ejecución

Sobre el commit 3ac010e pasan los 1.046 tests (218 de Dominio, 329 de Servicios, 185 de Repositorios, 301 de WebAPI y 13 de Fábrica), sin fallos ni tests omitidos.

Además de la suite, la API se probó de punta a punta con la colección de Postman de la carpeta Datos: 412 requests organizados por recurso y endpoint, todos con un test del código de estado y 234 que además verifican el errorCode del error. La última ejecución completa con el Collection Runner, contra la API local sobre SQL Server, dio 924 tests aprobados y ningún fallo (D-78). Las dos pruebas se complementan: los tests unitarios aíslan cada capa con dobles y señalan la regla exacta que se rompe, y la colección comprueba que las capas integradas, los filtros y la base real responden lo que dice la especificación de la API.

4.2 Cómo se midió la cobertura

La cobertura que vale para el proyecto es la que mide el check Test de GitHub Actions en cada Pull Request. El check corre todos los tests con coverlet.collector [3] y el .runsettings obligatorio del curso [5], y falla si algún assembly queda por debajo de 90 % de líneas o de ramas. Ese .runsettings excluye por archivo las migraciones (*Migrations/*.cs), Program.cs y los contextos de EF Core (*DbContext.cs y *Context.cs), y por atributo el código generado (GeneratedCodeAttribute). Se excluye lo que no tiene lógica propia que probar: las migraciones las genera dotnet ef a partir del modelo, Program.cs solo arranca la aplicación (lo que configura está en métodos con tests) y el contexto solo declara el mapeo. Las exclusiones son parámetros de la medición: el código no tiene ningún atributo ExcludeFromCodeCoverage.

Además, el equipo verificó la cobertura en Visual Studio con la extensión Fine Code Coverage, que no usa el .runsettings y mide todo el código, incluidos los proyectos de prueba. Sin exclusiones, Repositorios da 15,5 % de líneas (750 de 4.815) y 30,2 % de ramas (132 de 436), y WebAPI da 97,5 % de líneas (440 de 451) y 95,4 % de ramas (42 de 44). Todo lo que falta está en los archivos que excluye el .runsettings: las 4.065 líneas y 304 ramas sin cubrir de Repositorios son de las migraciones, y las 11 líneas y 2 ramas de WebAPI son de Program.cs. Al descontar esos archivos, los números coinciden con los del check: Repositorios y WebAPI quedan en 100 % de líneas y de ramas, y Servicios da 100 % de líneas y 99,6 % de ramas (269 de 270) en las dos herramientas. Las capturas están en el Anexo B (Ilustraciones 8 y 9).

4.3 Resultado

El check Test de GitHub Actions deja en cada Pull Request un comentario con la cobertura de cada assembly. En el PR #48, el último antes de la entrega, informa 100 % de líneas en los nueve assemblies y 99,95 % de ramas en total, por encima del 90 % que el check exige a cada uno (Ilustración 4). El único desvío es una rama de SorteoAleatorio.cs, el envoltorio de Random que en cada corrida ejecuta una sola de sus dos ramas, por lo que Servicios da 99,57 % de ramas.

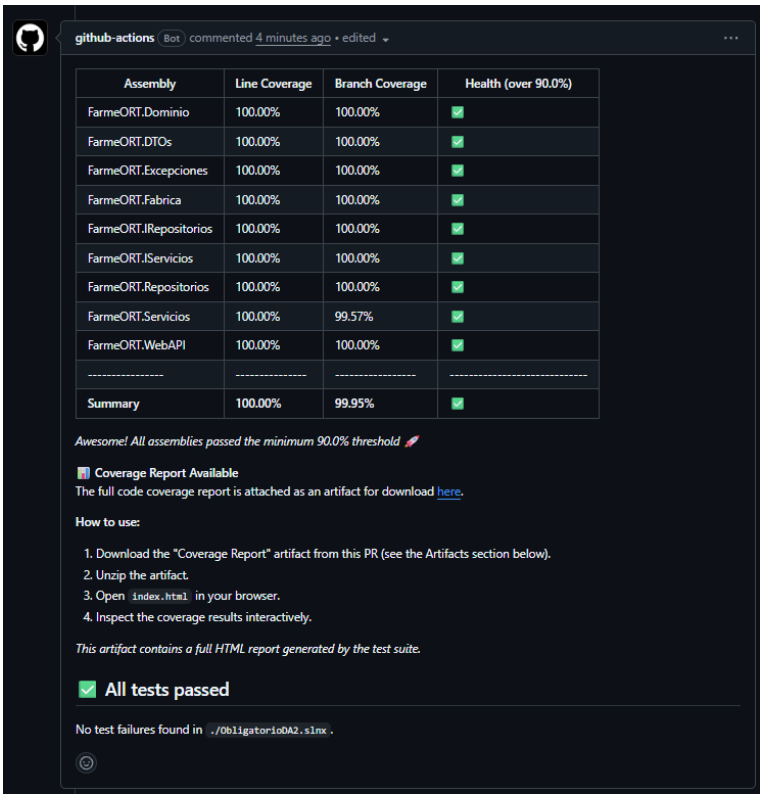


Ilustración 4. Cobertura por assembly informada por el check Test de GitHub Actions en el PR #48

Antes de llegar a estos números había dos desvíos que sí eran testeables y se corrigieron en el PR #36 en lugar de excluirlos: un test verifica que la base asigna un id propio a cada lanzamiento (1e4fd90) y un REFACTOR quitó una rama imposible de la comparación con un DateOnly? (d74d6ba). El 100 % es una consecuencia de TDD y no un objetivo en sí: no se escribe código de producción sin un test que lo pida. Los assemblies sin lógica (IRepositorios, IServicios, DTOs) también dan 100 % porque los tests usan sus tipos, pero ese número no informa sobre la calidad de las pruebas.

Fine Code Coverage, que mide sin exclusiones, da valores altos en todos los assemblies salvo

Repositorios, por las migraciones. El Anexo B muestra esa medición y explica por qué el porcentaje sube al excluirlas.

4.4 Calidad de las pruebas más allá de la cobertura

- Cada test verifica una regla con un assertion concreto: la excepción exacta (Assert.ThrowsExactly), el estado del objeto guardado o el valor calculado. Los tests de ELO usan DataRow con resultados de la fórmula de la letra: 1400 contra 1200 deja al ganador en 1408 y al perdedor en 1192.
- Los rechazos verifican también que no hubo efectos: que no se llamó a Agregar ni a Actualizar.
- La auditoría de legibilidad (D-68) reforzó tests débiles: un Verify del filtro que llega al repositorio reemplazó asserts sobre la lista vacía del mock (12c3c98), los estados forzados por reflexión se prepararon con operaciones del dominio (a4213b0) y se agregaron tests para reglas que no tenían uno.

La atomicidad y la concurrencia también tienen tests. La letra pide que fallar una batalla y cancelar un torneo tengan efectos atómicos, y una auditoría del 02/10/2026 reprodujo carreras entre pedidos simultáneos: dos inscripciones Aceptadas del mismo jugador, más Aceptadas que el cupo y una ronda generada dos veces (D-70). Cada caso empezó con un RED que reproduce la carrera: el test abre un segundo contexto sobre la misma base SQLite, intercala sus lecturas y guardados con los del repositorio bajo prueba y verifica que solo uno se confirma. Por ejemplo, Agregar_DosPedidosDelMismoJugadorLeenAntesDeGuardar_SoloUnoQuedaAceptado (RED fec00de, GREEN d45fdc1) y, en el fallo de una batalla, Guardar_EloConcurrente_RechazaFalloSinGuardarEfectosParciales (RED 9ac387e, GREEN 4afd02b), que cambia el ELO de un jugador desde otro contexto y comprueba que el fallo se rechaza sin dejar nada a medias: la batalla sigue Pendiente, ningún ELO cambia y la inscripción del perdedor sigue Aceptada. La corrección usa la concurrencia optimista de EF Core (una versión del torneo, D-70 y D-71) y responde 409 para que el cliente vuelva a consultar. Un test que dependiera de hilos y tiempos fallaría solo a veces; intercalar los pasos a mano hace que la carrera ocurra siempre igual, así que el test sigue siendo repetible.

5. Clean Code: por qué lo cumplimos

Para el equipo, Clean Code no es una lista para revisar al final: es lo que permite que el código de tres personas se lea como si lo hubiera escrito una sola, y que cualquier integrante pueda explicar cualquier parte en la defensa. Por eso una parte quedó automatizada y la otra es criterio de revisión en cada PR. El código sigue los capítulos 1 a 10 y 12 de Clean Code [4].

5.1 Reglas automáticas

Parte del estilo no depende de la disciplina de cada integrante: el .editorconfig y StyleCop.Analyzers de la configuración obligatoria del curso [5] marcan como error, entre otras cosas, no usar primary constructors para inyectar dependencias, namespaces que no son file-scoped, bloques sin llaves, usings sin usar y parámetros o miembros privados sin usar. El build falla si se rompe una de esas reglas, y el check Analysis corre dotnet format --verify-no-changes en cada PR. Ninguna regla se desactivó con #pragma, SuppressMessage ni cambios de severidad.

La ventaja es que las revisiones discuten diseño y reglas de negocio, no espacios ni llaves.

5.2 Cómo se aplica cada tema y qué se gana

La Tabla 5 resume, por tema del libro, qué se ve en el código y qué ventaja trae, y a continuación se describe la revisión de Clean Code previa a la entrega. El Anexo C desarrolla cada punto con más ejemplos.

Tabla 5. Clean Code en FarmeORT

Tema (capítulo)	En el código	Por qué / ventaja
Nombres (2)	Lenguaje de la letra: Inscripcion, EstadoTorneo.Abierto, TorneoNoAbiertoException; constantes K y EloMinimo; tests Metodo_Escenario_Resultado.	Una regla del código se compara con la letra sin traducir, y el nombre del test ya dice qué se espera.
Funciones (3)	Controllers de una a tres líneas; validaciones extraídas a métodos con nombre (ObtenerTorneoExistente).	Cada método se lee en un solo nivel de abstracción y en el orden de la letra.
Comentarios (4)	Solo explican el porqué, como el orden de las validaciones de la inscripción (D-67); la auditoría quitó los que no aportaban.	El comentario no queda desactualizado, porque explica una razón y no la mecánica.

Tema (capítulo)	En el código	Por qué / ventaja
Formato (5)	Lo imponen dotnet format y StyleCop; los métodos públicos van antes que los privados que usan.	La clase se lee de arriba hacia abajo y ningún PR discute estilo.
Objetos y estructuras (6)	Entidades con private set que cambian con métodos que validan (Inscripcion.Cancelar); DTOs sin comportamiento.	Nadie puede dejar una entidad en un estado inválido desde afuera, y la API nunca expone el dominio.
Manejo de errores (7)	Excepciones propias y un único FiltroExcepciones que las traduce a status y errorCode; ningún controller tiene try/catch.	Agregar un error es agregar una entrada al diccionario (OCP), y un error imprevisto nunca expone detalles internos.
Límites (8)	EF Core solo en Repositorios; reloj y azar detrás de TimeProvider e ISorteo; composición en FarmeORT.Fabrica.	Cambiar una librería o una fuente de datos afecta a un solo assembly.
Pruebas unitarias (9)	FIRST, un concepto por test, Arrange, Act y Assert separados.	Un test que falla señala una sola regla.
Clases (10)	Un servicio por caso de uso (ServicioLanzamientos, ServicioFallos…); internal sealed; puntos de variación detrás de interfaces.	Cada clase tiene una sola razón para cambiar y expone solo lo necesario.
Diseño emergente (12)	114 REFACTOR que quitan duplicación y acortan métodos largos (PR #43); ningún patrón que la letra no pida (D-51).	El diseño crece con los tests, sin complejidad anticipada.

Antes de la entrega se hizo además una revisión de Clean Code de todo src/ con tres criterios que se pueden medir: métodos de más de 20 líneas, números mágicos y métodos con más de 3 parámetros (PR #43). Se resolvió con 19 commits REFACTOR, uno por cambio, y ningún test se modificó: los 1.046 siguieron en verde después de cada commit, que es la prueba de que el comportamiento no cambió. Algunos resultados: OnModelCreating pasó de 65 a 11 líneas con una configuración por entidad (830b91b), ServicioFallos.Fallar pasó de 38 a 14 al extraer sus pasos con nombre (bb0bd0a), y el límite de paginado por defecto, que estaba repetido en cinco DTOs, quedó en una sola constante (3d454d0).

Lo que no se cambió también quedó registrado en el PR: Torneo.Modificar todavía recibe 9 parámetros, y los constructores de Torneo y Usuario reciben 8. Agruparlos en objetos parámetro cambia firmas públicas del dominio, muchos tests y el diagrama de clases, así que el equipo no

lo hizo a último momento y lo dejó como una mejora pendiente conocida.

5.3 Un ejemplo: el cálculo de ELO

El cálculo de ELO después del REFACTOR 71e0559 y 75a6c6f (Ilustración 5) reúne varios de esos puntos. El método público se lee como la regla de la letra: calcula el nuevo ELO del ganador y del perdedor, los dos con los valores previos. La fórmula está una sola vez, en CalcularNuevoElo, en lugar de repetirse para cada jugador; los números de la letra son constantes con nombre (K, EloMinimo, BaseLogistica y FactorDeEscala) y no valores sueltos, con un comentario que explica de dónde salen la base y la escala, y el piso de 100 es un if explícito. La clase implementa ICalculadorElo, así que otra fórmula se agrega con una implementación nueva sin tocar el código que la usa.

```
private const int K = 32;
private const int Victoria = 1;
private const int Derrota = 0;
private const int EloMinimo = 100;

// Con 400 puntos de diferencia, el de mayor ELO tiene 10 veces más chances de ganar (letra 1.10).
private const double BaseLogistica = 10;
private const double FactorDeEscala = 400.0;

public ResultadoElo Calcular(int eloGanador, int eloPerdedor)
{
   var nuevoEloGanador = CalcularNuevoElo(eloGanador, eloPerdedor, Victoria);
   var nuevoEloPerdedor = CalcularNuevoElo(eloPerdedor, eloGanador, Derrota);
   return new ResultadoElo(nuevoEloGanador, nuevoEloPerdedor);
}

private static int CalcularNuevoElo(int elo, int eloRival, int resultado)
{
   var puntajeEsperado = 1 / (1 + Math.Pow(BaseLogistica, (eloRival - elo) / FactorDeEscala));
   var nuevoElo = (int)Math.Round(elo + (K * (resultado - puntajeEsperado)));
   if(nuevoElo < EloMinimo)
   {
      return EloMinimo;
   }

   return nuevoElo;
}
```

Ilustración 5. Cálculo de ELO en CalculadorEloBase

6. Referencias bibliográficas

[1] J. Ferreiro, S. Luciardi y A. Bustamante, "M6CD-ID-274754-328344-302420," repositorio en GitHub, IngSoft-DA2, 2026. [Online]. Available: https://github.com/IngSoft-DA2/M6CD-ID-274754-328344-302420. Accessed on: Oct. 7, 2026.

[2] Universidad ORT Uruguay, Facultad de Ingeniería, "Obligatorio 1 – Diseño de Aplicaciones 2. FarmeORT: Batallas de Aura," Montevideo, Uruguay, Sep. 2026.

[3] coverlet-coverage, "coverlet," repositorio en GitHub. [Online]. Available: https://github.com/coverlet-coverage/coverlet. Accessed on: Oct. 7, 2026.

[4] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

[5] IngSoft-DA2, "DA2-Tecnologia, rama repo-configuration," repositorio en GitHub. [Online]. Available: https://github.com/IngSoft-DA2/DA2-Tecnologia/tree/repo-configuration. Accessed on: Oct. 7, 2026.

Anexo A. Evidencia de TDD por funcionalidad prioritaria

A.1 Inscripción a un torneo

La rama empezó por el controller y bajó al servicio y al dominio (ver Tabla 6).

Tabla 6. Fragmento del historial de la rama feature/inscripciones

Commit	Mensaje
ecc5f4b	test [RED] (FarmeORT.WebAPI.Tests/Controllers/): inscribe al jugador autenticado con 201 y Location
a9e842d	feat [GREEN] (FarmeORT.WebAPI/Controllers/InscripcionesController.cs): inscribe al jugador autenticado con 201 y Location
aea85a3	refactor [REFACTOR] (FarmeORT.WebAPI/Controllers/InscripcionesController.cs): unifica la lectura de la sesión
2904e6b	test [RED] (FarmeORT.Servicios.Tests/Inscripciones/): rechaza inscribirse con el cupo de aceptadas completo
2f46811	feat [GREEN] (FarmeORT.Servicios/Inscripciones/): rechaza inscribirse con el cupo de aceptadas completo
24c7ea5	refactor [REFACTOR] (FarmeORT.Dominio/Inscripciones/Inscripcion.cs): unifica el control de que la inscripción siga aceptada

Un ciclo completo es el del cupo de inscripciones aceptadas. El commit RED 2904e6b agrega el test de la Ilustración 6, la excepción TorneoCompletoException y solo la firma de ContarAceptadas en IRepositorioInscripciones; el test falla porque el servicio todavía no consulta el cupo.

```
[TestMethod]
public void Inscribir_CupoDeAceptadasCompleto_LanzaTorneoCompletoSinGuardar()
{
    ConfigurarInscripcionValida();
    _repositorioInscripciones.Setup(r => r.ContarAceptadas(TorneoId)).Returns(4);

    Assert.ThrowsExactly<TorneoCompletoException>(() => _servicio.Inscribir(JugadorId, new
```

```
InscripcionRequest { TorneoId = TorneoId }));
    _repositorioInscripciones.Verify(r => r.Agregar(It.IsAny<Inscripcion>()), Times.Never);
}
```

Ilustración 6. Test del commit RED 2904e6b

El commit GREEN 2f46811 agrega solo la validación de la Ilustración 7 y, como el mock es estricto, configura ContarAceptadas en el arrange común de los tests que ya existían.

```
if(repositorioInscripciones.ContarAceptadas(torneoId) >= torneo.Cupo)
{
    throw new TorneoCompletoException();
}
```

Ilustración 7. Validación agregada en el commit GREEN 2f46811

La regla quedó repartida según quién tiene la información: el estado del torneo y las skills obligatorias los controla la entidad Inscripcion al construirse; el duplicado y el cupo, el servicio, porque necesitan consultar las otras inscripciones guardadas.

A.2 Demás funcionalidades prioritarias

La Tabla 7 resume, para las demás funcionalidades, ciclos RED y GREEN representativos y el REFACTOR más relevante.

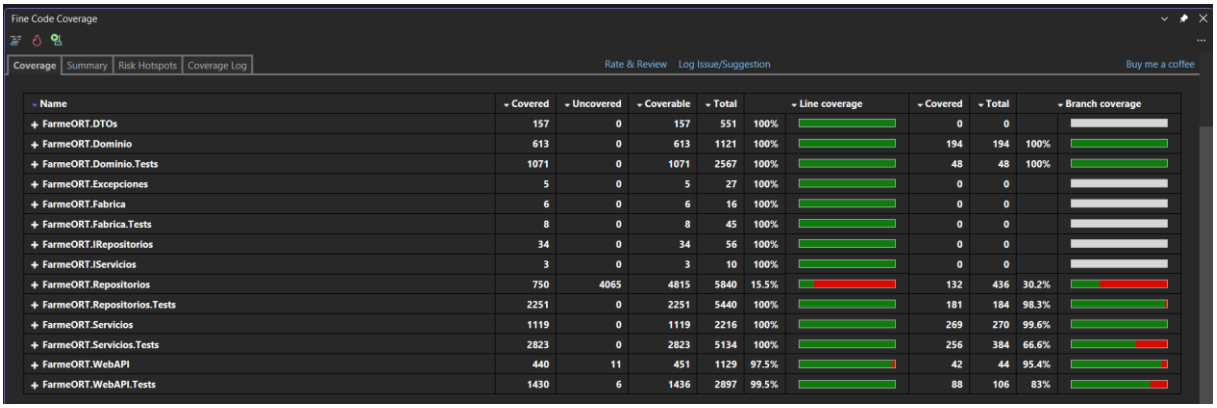
Tabla 7. Ciclos y refactors representativos de las demás funcionalidades prioritarias

Funcionalidad	**Ciclos RED → GREEN**	**REFACTOR destacado**
Catálogo de skills	fab4f7b → 275f56e: crea la skill con 201 y Location. ef706bf → 20ec93b: solo el administrador mantiene el catálogo.	ec26d51: el constructor reutiliza Modificar, así alta y modificación validan igual.
Lanzar una skill	bd497d3 → 5ed4057: empieza el jugador de mayor ELO. 93da606 → 14849c3: registra el lanzamiento y alterna el turno.	b452842: separa la validación del registro del lanzamiento.

Fallar una batalla y ELO	01ea0b6 → f7e4668: calcula ambos ELO con los valores previos. bb9fe47 → 5b57213: piso de 100. f374c54 → 05c5305: exige rol organizador.	71e0559: extrae el cálculo del ELO de un jugador. 04c3eb8: un test por efecto del fallo.
Generar rondas	f31f35f → 0424654: desempate determinístico por id. 8f553df → 132bef3: la primera ronda pasa el torneo a En curso.	323ff9d: separa la primera ronda de las siguientes.
Finalizar el torneo	44e8779 → 85f137f: devuelve al campeón. 29e0c0e → 2b04d28: rechaza al organizador no asignado.	2fb4f59: comparte en PermisosDelTorneo el control de permisos.

Anexo B. Cobertura sin exclusiones

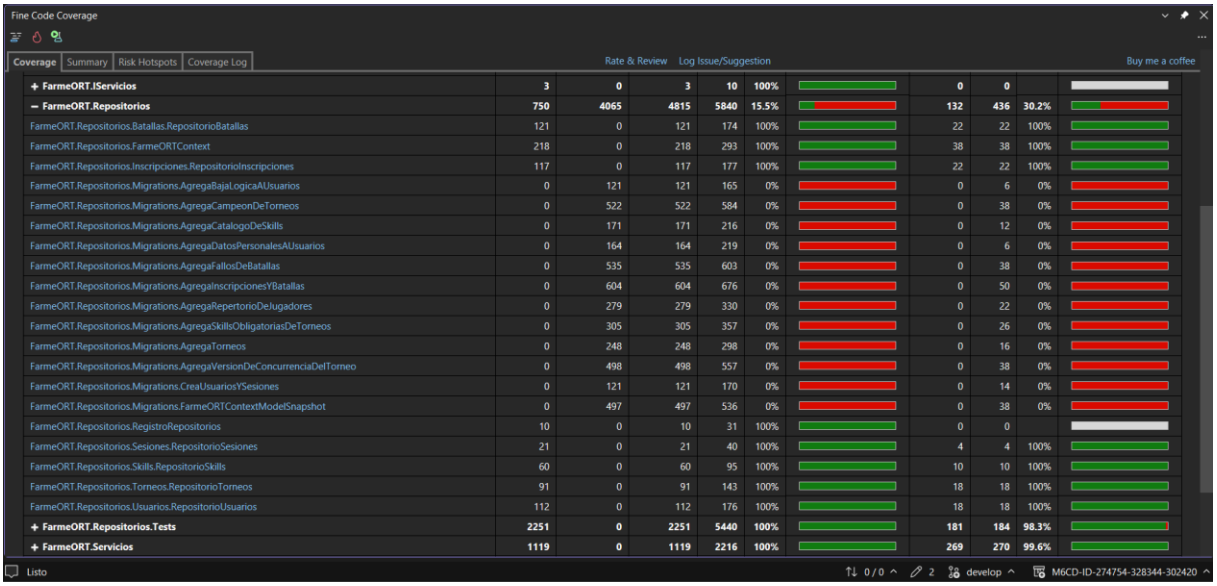
La Ilustración 8 muestra la cobertura que mide Fine Code Coverage en Visual Studio. A diferencia del check, no aplica el .runsettings: mide todo el código, incluidas las migraciones, Program.cs y los propios proyectos de prueba (las filas que terminan en .Tests, que no forman parte del criterio del curso). Los assemblies de producción dan entre 97,5 % y 100 % de líneas, salvo FarmeORT.Repositorios, que da 15,5 % de líneas y 30,2 % de ramas.



Name	Covered	Uncovered	Coverable	Total	Line coverage		Covered	Total	Branch coverage	
FarmeORT.DTOs	157	0	157	551	100%		0	0		
FarmeORT.Dominio	613	0	613	1121	100%		194	194	100%	
FarmeORT.Dominio.Tests	1071	0	1071	2567	100%		48	48	100%	
FarmeORT.Excepciones	5	0	5	27	100%		0	0		
FarmeORT.Fabrica	6	0	6	16	100%		0	0		
FarmeORT.Fabrica.Tests	8	0	8	45	100%		0	0		
FarmeORT.IRepositorios	34	0	34	56	100%		0	0		
FarmeORT.IServicios	3	0	3	10	100%		0	0		
FarmeORT.Repositorios	750	4065	4815	5840	15.5%		132	436	30.2%	
FarmeORT.Repositorios.Tests	2251	0	2251	5440	100%		181	184	98.3%	
FarmeORT.Servicios	1119	0	1119	2216	100%		269	270	99.6%	
FarmeORT.Servicios.Tests	2823	0	2823	5134	100%		256	384	66.6%	
FarmeORT.WebAPI	440	11	451	1129	97.5%		42	44	95.4%	
FarmeORT.WebAPI.Tests	1430	6	1436	2897	99.5%		88	106	83%	

Ilustración 8. Cobertura medida por Fine Code Coverage, sin exclusiones

La Ilustración 9 abre FarmeORT.Repositorios por clase. Los repositorios, el contexto y el registro de dependencias están al 100 %; todo lo que falta es de las clases del espacio de nombres Migrations, que dan 0 %.



FarmeORT.IServicios	3	0	3	10	100%		0	0		
FarmeORT.Repositorios	750	4065	4815	5840	15.5%		132	436	30.2%	
FarmeORT.Repositorios.Batallas.RepositorioBatallas	121	0	121	174	100%		22	22	100%	
FarmeORT.Repositorios.FarmeORTContext	218	0	218	293	100%		38	38	100%	
FarmeORT.Repositorios.Inscripciones.RepositorioInscripciones	117	0	117	177	100%		22	22	100%	
FarmeORT.Repositorios.Migrations.AgregaBajaLogicaAUsuarios	0	121	121	165	0%		0	6	0%	
FarmeORT.Repositorios.Migrations.AgregaCampeonDeTorneos	0	522	522	584	0%		0	38	0%	
FarmeORT.Repositorios.Migrations.AgregaCatalogoDeSkills	0	171	171	216	0%		0	12	0%	
FarmeORT.Repositorios.Migrations.AgregaDatosPersonalesAUsuarios	0	164	164	219	0%		0	6	0%	
FarmeORT.Repositorios.Migrations.AgregaFallosDeBatallas	0	535	535	603	0%		0	38	0%	
FarmeORT.Repositorios.Migrations.AgregaInscripcionesYBatallas	0	604	604	676	0%		0	50	0%	
FarmeORT.Repositorios.Migrations.AgregaRepertorioDeJugadores	0	279	279	330	0%		0	22	0%	
FarmeORT.Repositorios.Migrations.AgregaSkillsObligatoriasDeTorneos	0	305	305	357	0%		0	26	0%	
FarmeORT.Repositorios.Migrations.AgregaTorneos	0	248	248	298	0%		0	16	0%	
FarmeORT.Repositorios.Migrations.AgregaVersionDeConcurrenciaDelTorneo	0	498	498	557	0%		0	38	0%	
FarmeORT.Repositorios.Migrations.CreaUsuariosYSesiones	0	121	121	170	0%		0	14	0%	
FarmeORT.Repositorios.Migrations.FarmeORTContextModelSnapshot	0	497	497	536	0%		0	38	0%	
FarmeORT.Repositorios.RegistroRepositorios	10	0	10	31	100%		0	0		
FarmeORT.Repositorios.Sesiones.RepositorioSesiones	21	0	21	40	100%		4	4	100%	
FarmeORT.Repositorios.Skills.RepositorioSkills	60	0	60	95	100%		10	10	100%	
FarmeORT.Repositorios.Torneos.RepositorioTorneos	91	0	91	143	100%		18	18	100%	
FarmeORT.Repositorios.Usuarios.RepositorioUsuarios	112	0	112	176	100%		18	18	100%	
FarmeORT.Repositorios.Tests	2251	0	2251	5440	100%		181	184	98.3%	
FarmeORT.Servicios	1119	0	1119	2216	100%		269	270	99.6%	

Ilustración 9. Detalle de FarmeORT.Repositorios en Fine Code Coverage

Por qué sube al excluir las migraciones. El porcentaje de cobertura es el cociente entre las

líneas cubiertas y las líneas que se pueden cubrir. Las migraciones suman 4.065 líneas y 304 ramas, y ningún test las ejecuta: las genera dotnet ef a partir del modelo y solo corren con dotnet ef database update, mientras que los tests de repositorio crean la base SQLite en memoria con EnsureCreated, que arma el esquema directamente desde el modelo. Entran en el denominador, pero nunca en el numerador. Con ellas, Repositorios da 750 de 4.815 líneas (15,5 %) y 132 de 436 ramas (30,2 %).

Al excluirlas, como hace el .runsettings del check, el numerador no cambia y el denominador baja a las líneas y ramas del código escrito por el equipo: 750 de 750 líneas y 132 de 132 ramas, es decir, 100 % en las dos. Lo mismo pasa en WebAPI con Program.cs, que tiene las 11 líneas y 2 ramas sin cubrir: 440 de 451 líneas pasan a 440 de 440, y 42 de 44 ramas a 42 de 42. La exclusión no esconde código con lógica sin probar: saca de la cuenta código que genera la herramienta, y todo lo que el equipo escribió queda medido.

En resumen, la medición sin exclusiones no llega al 100 % por los tres elementos de la Tabla 8, ninguno con una regla de negocio que probar.

Tabla 8. Elementos no cubiertos en la medición sin exclusiones

Qué no se cubre	**Efecto en la medición**	**Por qué no lleva test**
Migraciones de EF Core (Migrations/*.cs)	4.065 líneas y 304 ramas; por eso Repositorios da 15,5 % de líneas y 30,2 % de ramas.	Las genera dotnet ef a partir del modelo y no tienen lógica propia.
Program.cs	11 líneas y 2 ramas; WebAPI da 97,5 % de líneas y 95,4 % de ramas.	Solo arranca la aplicación; lo que configura está en métodos con tests (RegistroWebApiTests y RegistroDependenciasTests).
SorteoAleatorio.cs	1 de sus 2 ramas; Servicios da 99,6 % de ramas (269 de 270).	Envuelve Random y en cada corrida ejecuta una sola rama; quién empieza ante un empate se prueba en BatallaTests con el doble SorteoPreparado.

El contexto de EF (FarmeORTContext.cs) no hace falta excluirlo: los tests de repositorio lo cubren al 100 %.

Anexo C. Clean Code en detalle

C.1 Nombres y funciones

Los nombres están en el lenguaje del dominio de la letra (Inscripcion, Batalla, Lanzamiento, EstadoTorneo.Abierto), así que una regla del código se compara con la letra sin traducir. Dicen la intención: ObtenerTorneoExistente deja claro que lanza si no existe, EstaVigente encapsula "aceptada en un torneo que no terminó" y cada excepción nombra la regla que se incumplió (TorneoNoAbiertoException, TurnoIncorrectoException). Los números de la letra son constantes con nombre (K, EloMinimo, BaseLogistica, FactorDeEscala, LanzamientosPorJugador) y los tests siguen el formato Metodo_Escenario_Resultado.

Las funciones son cortas y tienen un solo nivel de abstracción: los métodos de los controllers tienen de una a tres líneas, y las validaciones largas se extrajeron a métodos con nombre.

C.2 Comentarios y formato

Los comentarios explican el porqué y no repiten el código; por ejemplo, en ServicioInscripciones: "El estado va primero, como en la letra (1.5): un torneo En curso tiene el cupo lleno, pero el motivo del rechazo es que ya no admite inscripciones". La auditoría (D-68) quitó los comentarios que no aportaban, como marcas de autoría o referencias a reviews ya resueltos (2165049). El formato es uniforme porque lo impone dotnet format, y en las entidades los métodos públicos van antes que los privados que usan, para leer la clase de arriba hacia abajo.

C.3 Objetos, manejo de errores y límites

Las entidades esconden sus datos y exponen comportamiento: las propiedades tienen private set y el estado cambia solo con métodos que validan la regla (Inscripcion.Cancelar, Batalla.Lanzar); la lista de lanzamientos se expone como IReadOnlyCollection. Los DTOs son estructuras de datos sin comportamiento y la API nunca expone entidades. Los errores se informan con excepciones, y un único filtro global, FiltroExcepciones, las traduce a código de estado y errorCode, así que ningún controller tiene try/catch; un error no previsto devuelve 500 con un mensaje fijo. El código de terceros queda en los bordes: EF Core vive solo en Repositorios, detrás de IRepositorios; el reloj y el azar entran por TimeProvider e ISorteo, y la composición se resuelve en FarmeORT.Fabrica (D-34).

C.4 Pruebas unitarias y clases

Las pruebas son rápidas (los 1.046 tests corren en segundos), independientes, repetibles, auto-validadas y escritas antes del código, y cada una prueba un concepto. Las clases son chicas y tienen una sola razón para cambiar: en lugar de un servicio de batallas grande hay uno por caso de uso (ServicioLanzamientos, ServicioFallos, ServicioRondas, ServicioFinalizaciones). Reciben interfaces por primary constructor, y servicios, repositorios y filtros son internal sealed, lo que reduce el acoplamiento. Las reglas que la letra anticipa que pueden cambiar (otras fórmulas de ELO, otros emparejamientos) están detrás de una interfaz.

C.5 Diseño emergente y auditoría de legibilidad

Los REFACTOR siguieron las reglas del diseño simple: quitaron duplicación (7cff488, 24c7ea5, 2fb4f59) y no agregaron patrones ni librerías que la letra no pedía (por ejemplo, no hay una jerarquía State para los cuatro estados de la batalla, D-51). Antes de la entrega, el equipo revisó todo el código para que un tercero lo entienda con el menor esfuerzo (D-68, PR #30 y #32): reemplazó construcciones idiomáticas poco conocidas (?? throw, switch expressions, patrones relacionales) por if explícitos, sin cambiar el comportamiento, y los dos errores que encontró se corrigieron con su ciclo RED y GREEN.